

Milestone 2 - Avery Keuben (30170731)

Project URL: <https://csgit.ualgary.ca/avery.keuben/cpsc-411>

Git Tag: `milestone-2`

AI Disclaimer: AI was not used in the completion of this project.

Self Assessment: Outstanding performance

Self Assessment Evidence

Code Qualities

1. *Good error and warning messages output.* The included testing system ensures that my compiler outputs the same warnings and errors as the reference compiler on all test cases. They include the line numbers and a description provided by antlr4 of the issue. As described in the specification, the messages “should convey at least the same information [as the reference compiler]” which mine does exactly.
2. *Predictable, reliable, and tested:* The compiler outputs the exact same syntax tree as the reference compiler (using the included testing system, `make test`).

Development Practices

3. *Incremental development.* See git history. Each commit is atomic, changing or adding a single change per commit.
4. *Documentation of code changes: good commit messages.* See git history. Each commit accurately describes the content of the commit.

Code Qualities

1. *Code is readable:*
 - Shaping rules are split into separate categories, with spare lines separating subcategories into readable chunks.
 - Parser related files have been split into multiple different files (`parser.py`, `errors.py`, `shaping.py`, `shape.py`) to separate different components and make single files easier to take in and understand.
2. *Code is well documented:* Comments exist for non-obvious implementation details, including:
 - various shaping rules
 - overridden functions in the ASTShaper to perform some light post-processing, including why it is necessary to perform these extra steps, as well as how they are performed.
 - Doc comments are added to each class method (Lexer and Parser) to ensure understanding of the usage of each method.

3. *Code is modular and well-structured.* As with the lexer (see milestone 1 document `./milestone.md` in git repo) the parser is also wrapped in a class. This allows for a custom interface into the parser and separates the antlr4 code from our python code. This allows for the antlr4 parser to be replaced in the future without significant changes to the API of the parser. Further, as described above, the parser and lexer were split into multiple different files to separate different sections to make reading the code trivial.
4. *Code does not contain unnecessary duplication.* Common functionality between the lexer and parser are located in the `src/core` directory, and is not duplicated for each of the lexer and parser.
5. *Code is consistent, e.g., with respect to variable and function naming conventions.*
 - All variables and functions use camel case (e.g. `parserFromLexer`)
 - Python traditionally uses snake case, but antlr4 uses camel case (e.g. `symbolicNames`), so I decided to remain consistent with the core library we are using.
 - All classes are named with pascal case (e.g. `Parser` or `ParserErrorListener`)
6. *Speed* See run output. Code runs in a reasonable amount of time. The parser doesn't introduce any loops or recursion (outside of libraries) and makes use of the provided libraries which are assumed to be efficient.
7. *Code respects the environment in terms of the implementation language*
 - No additional libraries were used for the project, besides ANTLR4 and `cpsc411`, which is required. Made use of ANTLR4 and `cpsc411` in the intended way, just with a small wrapper around it.
 - Used the idomatic main function in python, including using `if __name__ == "__main__": main()` to ensure the main function is only ever called if it is the intended file to run.
 - Used try-except in case of an unknown file instead of checking if the file exists first.
8. *Code respects the environment in terms of tool usage:*
 - Used the provided grammar file with only small tweaks as recommended by the test cases, or to differentiate `globalVariableDeclarations` from `local variableDeclarations`.
 - Used `ASTShaper` in the intended way, only branching out to overriding the `nonterminal` function of the `Shaper` class when the shaping tools could not solve the problem in question
- 9.

Relationship with Runtime Environment

1. *Consistent with the environment: error and warning messages sent to `stderr`, regular output sent to `stdout`, exit status code set appropriately.*
 - Errors and warnings are sent to `stderr` (see run output)
 - Lexed tokens are sent to `stdout` (see run output)
 - Exiting due to a parsing error, lexing error, invalid command line

- usage, or unknown file results in an exit code of 1. (see run output)
- All error messages, warning messages, exit codes, and output messages are equivalent to that of the reference compiler (see custom unit testing output)
2. *No extraneous garbage in the output.*
 - Output is not colourized (see output)
 - Output matches character for character with the reference compiler.
 3. *Easy to build, and building without errors or warnings.*
 - No errors or warnings (see run output)
 - Building is easy using the provided make file (just run `make`)
 4. *Easy to run. Since the compiler's audience is you and the TA, it should be easy to run in the sense that it works to spec.*
 - Works with the provided `run` command.
 - Upon a fresh clone, all that is necessary is to run `make` and then run the main file `./main.py <input>` or `python main.py <input>`
 - You can easily run the provided unit tests with `make test`

Run Output

Run started on Wed Feb 25 00:51:50 2026

Hostname: csx2.uc.ualgary.ca

Running make

% /usr/bin/make

STDOUT:
make: Nothing to be done for 'all'.

STDERR:

RETURN CODE: 0

Locating J-- compiler

Compiler will be run as /usr/bin/python3 main.py

Test: too few arguments |

% /usr/bin/python3 main.py

STDOUT:

STDERR:
Usage: python3 main.py file.j--

RETURN CODE: 1

Test: too many arguments

% /usr/bin/python3 main.py /dev/null /dev/zero

STDOUT:

STDERR:
Usage: python3 main.py file.j--

RETURN CODE: 1

Test: nonexistent file

% /usr/bin/python3 main.py /i/do/not/exist

STDOUT:

STDERR:
/i/do/not/exist: No such file or directory

RETURN CODE: 1

Test: global variable declarations

```
% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t01
```

STDOUT:

```
program (lineno=4)
  globVarDecl (lineno=4)
    int (attr='int', lineno=4)
    id (attr='foo', lineno=4)
  globVarDecl (lineno=5)
    bool (attr='boolean', lineno=5)
    id (attr='bar', lineno=5)
```

STDERR:

RETURN CODE: 0

Test: syntax error

```
% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t02
```

STDOUT:

STDERR:

error: no viable alternative at input 'intqwertyint' at or near line 5

RETURN CODE: 1

Test: more and more complex global declarations

```
% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t03
```

STDOUT:

```
program (lineno=3)
  globVarDecl (lineno=3)
    int (attr='int', lineno=3)
    id (attr='foo', lineno=3)
  globVarDecl (lineno=4)
    bool (attr='boolean', lineno=4)
```

```

        id (attr='bar', lineno=4)
funcDecl (lineno=6)
    void (attr='void', lineno=6)
    id (attr='f1', lineno=6)
    formals (lineno=6)
    block (lineno=7)
        returnStmt (lineno=7)
funcDecl (lineno=10)
    int (attr='int', lineno=10)
    id (attr='f2', lineno=10)
    formals (lineno=10)
        formal (lineno=10)
            int (attr='int', lineno=10)
            id (attr='i', lineno=10)
    block (lineno=12)
        returnStmt (lineno=12)
            id (attr='i', lineno=12)
mainDecl (lineno=15)
    void (lineno=15)
    id (attr='main', lineno=15)
    formals (lineno=15)
    block (lineno=16)
funcDecl (lineno=18)
    bool (attr='boolean', lineno=18)
    id (attr='f3', lineno=18)
    formals (lineno=18)
        formal (lineno=18)
            int (attr='int', lineno=18)
            id (attr='i', lineno=18)
        formal (lineno=18)
            bool (attr='boolean', lineno=18)
            id (attr='b', lineno=18)
    block (lineno=20)
        returnStmt (lineno=20)
            false (attr='false', lineno=20)

```

STDERR:

RETURN CODE: 0

Test: the main problem

```
% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t04
```

STDOUT:

```
program (lineno=9)
  mainDecl (lineno=9)
    void (lineno=9)
    id (attr='startMeUp', lineno=9)
    formals (lineno=9)
      formal (lineno=9)
        int (attr='int', lineno=9)
        id (attr='error', lineno=9)
    block (lineno=10)
```

STDERR:

RETURN CODE: 0

Test: dangling else

```
% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t05
```

STDOUT:

```
program (lineno=3)
  mainDecl (lineno=3)
    void (lineno=3)
    id (attr='main', lineno=3)
    formals (lineno=3)
    block (lineno=4)
      ifStmt (lineno=4)
        true (attr='true', lineno=4)
      ifElseStmt (lineno=5)
        true (attr='true', lineno=5)
        nullStmt (lineno=6)
        nullStmt (lineno=8)
```

STDERR:

RETURN CODE: 0

Test: local declarations et al. |

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t06

STDOUT:

```
program (lineno=7)
  mainDecl (lineno=7)
    void (lineno=7)
    id (attr='main', lineno=7)
    formals (lineno=7)
    block (lineno=8)
      varDecl (lineno=8)
        bool (attr='boolean', lineno=8)
        id (attr='x', lineno=8)
      whileStmt (lineno=9)
        id (attr='x', lineno=9)
        block (lineno=10)
          varDecl (lineno=10)
            int (attr='int', lineno=10)
            id (attr='foo', lineno=10)
          breakStmt (lineno=11)
          nullStmt (lineno=12)
          nullStmt (lineno=13)
          nullStmt (lineno=14)
          block (lineno=15)
          block (lineno=17)
            varDecl (lineno=17)
              int (attr='int', lineno=17)
              id (attr='bar', lineno=17)
          breakStmt (lineno=19)
```

STDERR:

RETURN CODE: 0

Test: precedence |

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t07

STDOUT:

```
program (lineno=3)
```



```

mainDecl (lineno=3)
  void (lineno=3)
  id (attr='main', lineno=3)
  formals (lineno=3)
  block (lineno=5)
    exprStmt (lineno=5)
      ASSIGN (lineno=5)
        id (attr='x', lineno=5)
      ADD (lineno=5)
        number (attr='2', lineno=5)
      MUL (lineno=5)
        number (attr='3', lineno=5)
        number (attr='5', lineno=5)
    exprStmt (lineno=6)
      ASSIGN (lineno=6)
        id (attr='x', lineno=6)
      MUL (lineno=6)
        ADD (lineno=6)
          number (attr='2', lineno=6)
          number (attr='3', lineno=6)
        number (attr='5', lineno=6)

```

STDERR:

RETURN CODE: 0

Test: associativity

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t08

STDOUT:

```

program (lineno=3)
  mainDecl (lineno=3)
    void (lineno=3)
    id (attr='main', lineno=3)
    formals (lineno=3)
    block (lineno=4)
      exprStmt (lineno=4)
        ASSIGN (lineno=4)
          id (attr='x', lineno=4)
        DIV (lineno=4)
          DIV (lineno=4)

```

```

        DIV (lineno=4)
            number (attr='1', lineno=4)
            number (attr='2', lineno=4)
            number (attr='3', lineno=4)
            number (attr='4', lineno=4)
    exprStmt (lineno=5)
        ASSIGN (lineno=5)
            id (attr='a', lineno=5)
        ASSIGN (lineno=5)
            id (attr='b', lineno=5)
            id (attr='c', lineno=5)

```

STDERR:

RETURN CODE: 0

Test: unary minuses, so many unary minuses

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t09

STDOUT:

```

program (lineno=9)
    mainDecl (lineno=9)
        void (lineno=9)
        id (attr='main', lineno=9)
        formals (lineno=9)
        block (lineno=10)
            exprStmt (lineno=10)
                ASSIGN (lineno=10)
                    id (attr='x', lineno=10)
                SUB (lineno=10)
                    id (attr='a', lineno=10)
                    number (attr='-1', lineno=10)
            exprStmt (lineno=11)
                ASSIGN (lineno=11)
                    id (attr='x', lineno=11)
                SUB (lineno=11)
                    id (attr='a', lineno=11)
                    number (attr='-1', lineno=11)
            exprStmt (lineno=12)
                ASSIGN (lineno=12)
                    id (attr='x', lineno=12)

```

```

        SUB (lineno=12)
            id (attr='a', lineno=12)
            number (attr='-1', lineno=12)
    exprStmt (lineno=13)
        ASSIGN (lineno=13)
            id (attr='x', lineno=13)
            SUB (lineno=13)
                id (attr='a', lineno=13)
                number (attr='-1', lineno=19)
    exprStmt (lineno=20)
        ASSIGN (lineno=20)
            id (attr='x', lineno=20)
            SUB (lineno=20)
                id (attr='a', lineno=20)
                number (attr='-1', lineno=20)
    exprStmt (lineno=21)
        ASSIGN (lineno=21)
            id (attr='x', lineno=21)
            SUB (lineno=21)
                id (attr='a', lineno=21)
                UMINUS (lineno=21)
                    number (attr='-1', lineno=21)
    exprStmt (lineno=22)
        ASSIGN (lineno=22)
            id (attr='x', lineno=22)
            SUB (lineno=22)
                id (attr='a', lineno=22)
                UMINUS (lineno=22)
                    id (attr='b', lineno=22)
    exprStmt (lineno=23)
        ASSIGN (lineno=23)
            id (attr='x', lineno=23)
            UMINUS (lineno=23)
                UMINUS (lineno=23)
                    UMINUS (lineno=23)
                        UMINUS (lineno=23)
                            UMINUS (lineno=23)
                                id (attr='x', lineno=23)

```

STDERR:

RETURN CODE: 0

Test: stringing your parser along

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t10

STDOUT:

```
program (lineno=3)
  mainDecl (lineno=3)
    void (lineno=3)
    id (attr='mane', lineno=3)
    formals (lineno=3)
    block (lineno=7)
      exprStmt (lineno=7)
        funcCall (lineno=7)
          id (attr='prints', lineno=7)
          actuals (lineno=7)
            string (attr='"Hello, world!\\n"', lineno=7)
      exprStmt (lineno=8)
        funcCall (lineno=8)
          id (attr='printi', lineno=8)
          actuals (lineno=8)
            string (attr='"Hello, world!\\n"', lineno=8)
```

STDERR:

RETURN CODE: 0

Test: format's last theorem

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t11

STDOUT:

```
program (lineno=3)
  mainDecl (lineno=3)
    void (lineno=3)
    id (attr='Maine', lineno=3)
    formals (lineno=3)
    block (lineno=5)
      ifStmt (lineno=5)
        EQ (lineno=6)
```

```

        ADD (lineno=6)
            number (attr='2', lineno=6)
            number (attr='3', lineno=7)
            number (attr='5', lineno=8)
        block (lineno=9)

```

STDERR:

RETURN CODE: 0

Test: formalities and actualities

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t12

STDOUT:

```

program (lineno=3)
    funcDecl (lineno=3)
        void (attr='void', lineno=3)
        id (attr='foo', lineno=3)
        formals (lineno=3)
        block (lineno=3)
    funcDecl (lineno=5)
        bool (attr='boolean', lineno=5)
        id (attr='bar', lineno=5)
        formals (lineno=5)
            formal (lineno=5)
                int (attr='int', lineno=5)
                id (attr='a1', lineno=5)
            formal (lineno=5)
                bool (attr='boolean', lineno=5)
                id (attr='a2', lineno=5)
            formal (lineno=5)
                int (attr='int', lineno=5)
                id (attr='a3', lineno=5)
        block (lineno=5)
    funcDecl (lineno=8)
        int (attr='int', lineno=8)
        id (attr='baz', lineno=8)
        formals (lineno=8)
        block (lineno=9)
            exprStmt (lineno=9)
            funcCall (lineno=9)

```

```

id (attr='blarg', lineno=9)
actuals (lineno=10)
  ADD (lineno=10)
    number (attr='123', lineno=10)
    number (attr='456', lineno=10)
  funcCall (lineno=11)
    id (attr='garble', lineno=11)
    actuals (lineno=11)
      number (attr='789', lineno=11)
      id (attr='a', lineno=11)
      MUL (lineno=11)
        id (attr='b', lineno=11)
        id (attr='c', lineno=11)
    true (attr='true', lineno=12)
  funcCall (lineno=13)
    id (attr='greep', lineno=13)
    actuals (lineno=13)
      false (attr='false', lineno=13)
      string (attr='"eek"', lineno=13)

```

STDERR:

RETURN CODE: 0

Test: the perfect app

```
% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t13
```

STDOUT:
program (lineno=3)

STDERR:

RETURN CODE: 0

Test: syntax error?

```
% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/parse.t14
```

STDOUT:

```
program (lineno=5)
  mainDecl (lineno=5)
    void (lineno=5)
    id (attr='main', lineno=5)
    formals (lineno=5)
    block (lineno=6)
      exprStmt (lineno=6)
        number (attr='123', lineno=6)
```

STDERR:

RETURN CODE: 0

Test: a calculated move

% /usr/bin/python3 main.py /home/profs/aycock/411/TEST/ms2/gen.t18

STDOUT:

```
program (lineno=5)
  mainDecl (lineno=5)
    void (lineno=5)
    id (attr='calculator', lineno=5)
    formals (lineno=5)
    block (lineno=7)
      exprStmt (lineno=7)
        funcCall (lineno=7)
          id (attr='init', lineno=7)
          actuals (lineno=7)
      exprStmt (lineno=8)
        funcCall (lineno=8)
          id (attr='parser', lineno=8)
          actuals (lineno=8)
  globVarDecl (lineno=15)
    bool (attr='boolean', lineno=15)
    id (attr='_havechar', lineno=15)
  globVarDecl (lineno=16)
    int (attr='int', lineno=16)
    id (attr='_char', lineno=16)
  funcDecl (lineno=18)
    int (attr='int', lineno=18)
```

```

id (attr='getc', lineno=18)
formals (lineno=18)
block (lineno=20)
    ifStmt (lineno=20)
        id (attr='_havechar', lineno=20)
        block (lineno=21)
            exprStmt (lineno=21)
                ASSIGN (lineno=21)
                    id (attr='_havechar', lineno=21)
                    false (attr='false', lineno=21)
            returnStmt (lineno=22)
                id (attr='_char', lineno=22)
        returnStmt (lineno=24)
            funcCall (lineno=24)
                id (attr='getchar', lineno=24)
                actuals (lineno=24)
funcDecl (lineno=27)
    void (attr='void', lineno=27)
    id (attr='ungetc', lineno=27)
    formals (lineno=27)
        formal (lineno=27)
            int (attr='int', lineno=27)
            id (attr='ch', lineno=27)
    block (lineno=29)
        ifStmt (lineno=29)
            id (attr='_havechar', lineno=29)
            block (lineno=30)
                exprStmt (lineno=30)
                    funcCall (lineno=30)
                        id (attr='prints', lineno=30)
                        actuals (lineno=30)
                            string (attr='"Internal error: too many ungets!\\n"', lineno=30)
                exprStmt (lineno=31)
                    funcCall (lineno=31)
                        id (attr='halt', lineno=31)
                        actuals (lineno=31)
            exprStmt (lineno=33)
                ASSIGN (lineno=33)
                    id (attr='_havechar', lineno=33)
                    true (attr='true', lineno=33)
            exprStmt (lineno=34)
                ASSIGN (lineno=34)
                    id (attr='_char', lineno=34)
                    id (attr='ch', lineno=34)
globVarDecl (lineno=41)
    bool (attr='boolean', lineno=41)

```



```

        id (attr='_havetoken', lineno=41)
globVarDecl (lineno=42)
    int (attr='int', lineno=42)
    id (attr='_token', lineno=42)
globVarDecl (lineno=43)
    int (attr='int', lineno=43)
    id (attr='attr', lineno=43)
funcDecl (lineno=45)
    int (attr='int', lineno=45)
    id (attr='peek', lineno=45)
    formals (lineno=45)
    block (lineno=47)
        ifStmt (lineno=47)
            id (attr='_havetoken', lineno=47)
            returnStmt (lineno=48)
                id (attr='_token', lineno=48)
        exprStmt (lineno=50)
            ASSIGN (lineno=50)
                id (attr='_havetoken', lineno=50)
                true (attr='true', lineno=50)
        returnStmt (lineno=51)
            ASSIGN (lineno=51)
                id (attr='_token', lineno=51)
                funcCall (lineno=51)
                    id (attr='scanner', lineno=51)
                    actuals (lineno=51)
funcDecl (lineno=54)
    void (attr='void', lineno=54)
    id (attr='match', lineno=54)
    formals (lineno=54)
        formal (lineno=54)
            int (attr='int', lineno=54)
            id (attr='expect', lineno=54)
    block (lineno=56)
        ifStmt (lineno=56)
            NE (lineno=56)
                funcCall (lineno=56)
                    id (attr='peek', lineno=56)
                    actuals (lineno=56)
                    id (attr='expect', lineno=56)
            block (lineno=57)
                exprStmt (lineno=57)
                    funcCall (lineno=57)
                        id (attr='prints', lineno=57)
                        actuals (lineno=57)
                            string (attr='"Error: expected "', lineno=57)

```

```

    exprStmt (lineno=58)
      funcCall (lineno=58)
        id (attr='putc', lineno=58)
        actuals (lineno=58)
          id (attr='expect', lineno=58)
    exprStmt (lineno=59)
      funcCall (lineno=59)
        id (attr='prints', lineno=59)
        actuals (lineno=59)
          string (attr='"\n"', lineno=59)
    exprStmt (lineno=60)
      funcCall (lineno=60)
        id (attr='halt', lineno=60)
        actuals (lineno=60)
    exprStmt (lineno=62)
      ASSIGN (lineno=62)
        id (attr='_havetoken', lineno=62)
        false (attr='false', lineno=62)
funcDecl (lineno=65)
  int (attr='int', lineno=65)
  id (attr='scanner', lineno=65)
  formals (lineno=65)
  block (lineno=67)
    varDecl (lineno=67)
      int (attr='int', lineno=67)
      id (attr='ch', lineno=67)
    whileStmt (lineno=69)
      funcCall (lineno=69)
        id (attr='isspace', lineno=69)
        actuals (lineno=69)
          ASSIGN (lineno=69)
            id (attr='ch', lineno=69)
            funcCall (lineno=69)
              id (attr='getc', lineno=69)
              actuals (lineno=69)
            nullStmt (lineno=70)
          ifStmt (lineno=72)
            EQ (lineno=72)
              id (attr='ch', lineno=72)
              id (attr='EOF', lineno=72)
            returnStmt (lineno=72)
              id (attr='TK_EOF', lineno=72)
          ifStmt (lineno=73)
            EQ (lineno=73)
              id (attr='ch', lineno=73)
              id (attr='ASCII_NL', lineno=73)

```

```

        returnStmt (lineno=73)
            id (attr='TK_EOLN', lineno=73)
    ifStmt (lineno=74)
        EQ (lineno=74)
            id (attr='ch', lineno=74)
            id (attr='ASCII_PLUS', lineno=74)
        returnStmt (lineno=74)
            id (attr='TK_ADD', lineno=74)
    ifStmt (lineno=75)
        EQ (lineno=75)
            id (attr='ch', lineno=75)
            id (attr='ASCII_MINUS', lineno=75)
        returnStmt (lineno=75)
            id (attr='TK_SUB', lineno=75)
    ifStmt (lineno=76)
        EQ (lineno=76)
            id (attr='ch', lineno=76)
            id (attr='ASCII_STAR', lineno=76)
        returnStmt (lineno=76)
            id (attr='TK_MUL', lineno=76)
    ifStmt (lineno=77)
        EQ (lineno=77)
            id (attr='ch', lineno=77)
            id (attr='ASCII_SLASH', lineno=77)
        returnStmt (lineno=77)
            id (attr='TK_DIV', lineno=77)
    ifStmt (lineno=78)
        EQ (lineno=78)
            id (attr='ch', lineno=78)
            id (attr='ASCII_LPAREN', lineno=78)
        returnStmt (lineno=78)
            id (attr='TK_LPAREN', lineno=78)
    ifStmt (lineno=79)
        EQ (lineno=79)
            id (attr='ch', lineno=79)
            id (attr='ASCII_RPAREN', lineno=79)
        returnStmt (lineno=79)
            id (attr='TK_RPAREN', lineno=79)
    ifStmt (lineno=81)
        funcCall (lineno=81)
            id (attr='isdigit', lineno=81)
            actuals (lineno=81)
                id (attr='ch', lineno=81)
        block (lineno=82)
            exprStmt (lineno=82)
                ASSIGN (lineno=82)

```

```

        id (attr='attr', lineno=82)
        number (attr='0', lineno=82)
whileStmt (lineno=83)
    funcCall (lineno=83)
        id (attr='isdigit', lineno=83)
        actuals (lineno=83)
            id (attr='ch', lineno=83)
    block (lineno=84)
        exprStmt (lineno=84)
            ASSIGN (lineno=84)
                id (attr='attr', lineno=84)
            ADD (lineno=84)
                MUL (lineno=84)
                    id (attr='attr', lineno=84)
                    number (attr='10', lineno=84)
                SUB (lineno=84)
                    id (attr='ch', lineno=84)
                    id (attr='ASCII_0', lineno=84)
            exprStmt (lineno=85)
                ASSIGN (lineno=85)
                    id (attr='ch', lineno=85)
                funcCall (lineno=85)
                    id (attr='getc', lineno=85)
                    actuals (lineno=85)
exprStmt (lineno=87)
    funcCall (lineno=87)
        id (attr='ungetc', lineno=87)
        actuals (lineno=87)
            id (attr='ch', lineno=87)
    returnStmt (lineno=88)
        id (attr='TK_NUMBER', lineno=88)
exprStmt (lineno=91)
    funcCall (lineno=91)
        id (attr='prints', lineno=91)
        actuals (lineno=91)
            string (attr='"Error: invalid character.\\n"', lineno=91)
exprStmt (lineno=92)
    funcCall (lineno=92)
        id (attr='halt', lineno=92)
        actuals (lineno=92)
funcDecl (lineno=95)
    bool (attr='boolean', lineno=95)
    id (attr='isdigit', lineno=95)
    formals (lineno=95)
        formal (lineno=95)
            int (attr='int', lineno=95)

```

```

        id (attr='ch', lineno=95)
    block (lineno=97)
        returnStmt (lineno=97)
        AND (lineno=97)
            GE (lineno=97)
                id (attr='ch', lineno=97)
                id (attr='ASCII_0', lineno=97)
            LE (lineno=97)
                id (attr='ch', lineno=97)
                id (attr='ASCII_9', lineno=97)
funcDecl (lineno=100)
    bool (attr='boolean', lineno=100)
    id (attr='isspace', lineno=100)
    formals (lineno=100)
        formal (lineno=100)
            int (attr='int', lineno=100)
            id (attr='ch', lineno=100)
    block (lineno=102)
        returnStmt (lineno=102)
        OR (lineno=102)
            OR (lineno=102)
                EQ (lineno=102)
                    id (attr='ch', lineno=102)
                    id (attr='ASCII_SPACE', lineno=102)
                EQ (lineno=103)
                    id (attr='ch', lineno=103)
                    id (attr='ASCII_TAB', lineno=103)
            EQ (lineno=104)
                id (attr='ch', lineno=104)
                id (attr='ASCII_CR', lineno=104)
funcDecl (lineno=111)
    void (attr='void', lineno=111)
    id (attr='parser', lineno=111)
    formals (lineno=111)
    block (lineno=113)
        varDecl (lineno=113)
            int (attr='int', lineno=113)
            id (attr='result', lineno=113)
        whileStmt (lineno=115)
            NE (lineno=115)
                funcCall (lineno=115)
                    id (attr='peek', lineno=115)
                    actuals (lineno=115)
                    id (attr='TK_EOF', lineno=115)
                block (lineno=116)
                    exprStmt (lineno=116)

```

```

        ASSIGN (lineno=116)
            id (attr='result', lineno=116)
            funcCall (lineno=116)
                id (attr='E', lineno=116)
                actuals (lineno=116)
    exprStmt (lineno=117)
        funcCall (lineno=117)
            id (attr='match', lineno=117)
            actuals (lineno=117)
                id (attr='TK_EOLN', lineno=117)
    exprStmt (lineno=119)
        funcCall (lineno=119)
            id (attr='prints', lineno=119)
            actuals (lineno=119)
                string (attr='" = "', lineno=119)
    exprStmt (lineno=120)
        funcCall (lineno=120)
            id (attr='printi', lineno=120)
            actuals (lineno=120)
                id (attr='result', lineno=120)
    exprStmt (lineno=121)
        funcCall (lineno=121)
            id (attr='prints', lineno=121)
            actuals (lineno=121)
                string (attr='"\\n"', lineno=121)
funcDecl (lineno=125)
    int (attr='int', lineno=125)
    id (attr='E', lineno=125)
    formals (lineno=125)
    block (lineno=127)
        varDecl (lineno=127)
            int (attr='int', lineno=127)
            id (attr='l', lineno=127)
        varDecl (lineno=128)
            int (attr='int', lineno=128)
            id (attr='r', lineno=128)
        varDecl (lineno=129)
            int (attr='int', lineno=129)
            id (attr='token', lineno=129)
    exprStmt (lineno=131)
        ASSIGN (lineno=131)
            id (attr='l', lineno=131)
            funcCall (lineno=131)
                id (attr='T', lineno=131)
                actuals (lineno=131)
    whileStmt (lineno=132)

```

```

OR (lineno=132)
  EQ (lineno=132)
    funcCall (lineno=132)
      id (attr='peek', lineno=132)
      actuals (lineno=132)
      id (attr='TK_ADD', lineno=132)
  EQ (lineno=132)
    funcCall (lineno=132)
      id (attr='peek', lineno=132)
      actuals (lineno=132)
      id (attr='TK_SUB', lineno=132)
block (lineno=133)
  exprStmt (lineno=133)
    funcCall (lineno=133)
      id (attr='match', lineno=133)
      actuals (lineno=133)
      ASSIGN (lineno=133)
        id (attr='token', lineno=133)
        funcCall (lineno=133)
          id (attr='peek', lineno=133)
          actuals (lineno=133)
  exprStmt (lineno=134)
    ASSIGN (lineno=134)
      id (attr='r', lineno=134)
      funcCall (lineno=134)
        id (attr='T', lineno=134)
        actuals (lineno=134)
  ifElseStmt (lineno=136)
    EQ (lineno=136)
      id (attr='token', lineno=136)
      id (attr='TK_ADD', lineno=136)
    exprStmt (lineno=137)
      ASSIGN (lineno=137)
        id (attr='l', lineno=137)
        ADD (lineno=137)
          id (attr='l', lineno=137)
          id (attr='r', lineno=137)
    exprStmt (lineno=139)
      ASSIGN (lineno=139)
        id (attr='l', lineno=139)
        SUB (lineno=139)
          id (attr='l', lineno=139)
          id (attr='r', lineno=139)
  returnStmt (lineno=141)
    id (attr='l', lineno=141)
funcDecl (lineno=144)

```

```

int (attr='int', lineno=144)
id (attr='T', lineno=144)
formals (lineno=144)
block (lineno=146)
  varDecl (lineno=146)
    int (attr='int', lineno=146)
    id (attr='l', lineno=146)
  varDecl (lineno=147)
    int (attr='int', lineno=147)
    id (attr='r', lineno=147)
  varDecl (lineno=148)
    int (attr='int', lineno=148)
    id (attr='token', lineno=148)
  exprStmt (lineno=150)
    ASSIGN (lineno=150)
      id (attr='l', lineno=150)
      funcCall (lineno=150)
        id (attr='F', lineno=150)
        actuals (lineno=150)
  whileStmt (lineno=151)
    OR (lineno=151)
      EQ (lineno=151)
        funcCall (lineno=151)
          id (attr='peek', lineno=151)
          actuals (lineno=151)
          id (attr='TK_MUL', lineno=151)
      EQ (lineno=151)
        funcCall (lineno=151)
          id (attr='peek', lineno=151)
          actuals (lineno=151)
          id (attr='TK_DIV', lineno=151)
    block (lineno=152)
      exprStmt (lineno=152)
        funcCall (lineno=152)
          id (attr='match', lineno=152)
          actuals (lineno=152)
          ASSIGN (lineno=152)
            id (attr='token', lineno=152)
            funcCall (lineno=152)
              id (attr='peek', lineno=152)
              actuals (lineno=152)
      exprStmt (lineno=153)
        ASSIGN (lineno=153)
          id (attr='r', lineno=153)
          funcCall (lineno=153)
            id (attr='F', lineno=153)

```



```

        actuals (lineno=153)
    ifElseStmt (lineno=155)
        EQ (lineno=155)
            id (attr='token', lineno=155)
            id (attr='TK_MUL', lineno=155)
        exprStmt (lineno=156)
            ASSIGN (lineno=156)
                id (attr='l', lineno=156)
            MUL (lineno=156)
                id (attr='l', lineno=156)
                id (attr='r', lineno=156)
        exprStmt (lineno=158)
            ASSIGN (lineno=158)
                id (attr='l', lineno=158)
            DIV (lineno=158)
                id (attr='l', lineno=158)
                id (attr='r', lineno=158)
    returnStmt (lineno=160)
        id (attr='l', lineno=160)
funcDecl (lineno=163)
    int (attr='int', lineno=163)
    id (attr='F', lineno=163)
    formals (lineno=163)
    block (lineno=165)
        varDecl (lineno=165)
            int (attr='int', lineno=165)
            id (attr='result', lineno=165)
        varDecl (lineno=167)
            int (attr='int', lineno=167)
            id (attr='token', lineno=167)
        exprStmt (lineno=168)
            ASSIGN (lineno=168)
                id (attr='token', lineno=168)
            funcCall (lineno=168)
                id (attr='peek', lineno=168)
                actuals (lineno=168)
    ifElseStmt (lineno=170)
        EQ (lineno=170)
            id (attr='token', lineno=170)
            id (attr='TK_LPAREN', lineno=170)
        block (lineno=171)
            exprStmt (lineno=171)
                funcCall (lineno=171)
                    id (attr='match', lineno=171)
                    actuals (lineno=171)
                        id (attr='TK_LPAREN', lineno=171)

```

```

exprStmt (lineno=172)
  ASSIGN (lineno=172)
    id (attr='result', lineno=172)
    funcCall (lineno=172)
      id (attr='E', lineno=172)
      actuals (lineno=172)
exprStmt (lineno=173)
  funcCall (lineno=173)
    id (attr='match', lineno=173)
    actuals (lineno=173)
      id (attr='TK_RPAREN', lineno=173)
ifElseStmt (lineno=175)
  EQ (lineno=175)
    id (attr='token', lineno=175)
    id (attr='TK_SUB', lineno=175)
  block (lineno=176)
    exprStmt (lineno=176)
      funcCall (lineno=176)
        id (attr='match', lineno=176)
        actuals (lineno=176)
          id (attr='TK_SUB', lineno=176)
    exprStmt (lineno=177)
      ASSIGN (lineno=177)
        id (attr='result', lineno=177)
        UMINUS (lineno=177)
          funcCall (lineno=177)
            id (attr='F', lineno=177)
            actuals (lineno=177)
ifElseStmt (lineno=179)
  EQ (lineno=179)
    id (attr='token', lineno=179)
    id (attr='TK_NUMBER', lineno=179)
  block (lineno=180)
    exprStmt (lineno=180)
      funcCall (lineno=180)
        id (attr='match', lineno=180)
        actuals (lineno=180)
          id (attr='TK_NUMBER', lineno=180)
    exprStmt (lineno=181)
      ASSIGN (lineno=181)
        id (attr='result', lineno=181)
        id (attr='attr', lineno=181)
  block (lineno=184)
    exprStmt (lineno=184)
      funcCall (lineno=184)
        id (attr='prints', lineno=184)

```

```

                                actuals (lineno=184)
                                string (attr='Error: expected factor.\n', lineno=
exprStmt (lineno=185)
                                funcCall (lineno=185)
                                    id (attr='halt', lineno=185)
                                    actuals (lineno=185)
                                returnStmt (lineno=187)
                                    id (attr='result', lineno=187)
globVarDecl (lineno=194)
    int (attr='int', lineno=194)
    id (attr='EOF', lineno=194)
globVarDecl (lineno=196)
    int (attr='int', lineno=196)
    id (attr='TK_EOF', lineno=196)
globVarDecl (lineno=197)
    int (attr='int', lineno=197)
    id (attr='TK_EOLN', lineno=197)
globVarDecl (lineno=198)
    int (attr='int', lineno=198)
    id (attr='TK_NUMBER', lineno=198)
globVarDecl (lineno=199)
    int (attr='int', lineno=199)
    id (attr='TK_ADD', lineno=199)
globVarDecl (lineno=200)
    int (attr='int', lineno=200)
    id (attr='TK_SUB', lineno=200)
globVarDecl (lineno=201)
    int (attr='int', lineno=201)
    id (attr='TK_MUL', lineno=201)
globVarDecl (lineno=202)
    int (attr='int', lineno=202)
    id (attr='TK_DIV', lineno=202)
globVarDecl (lineno=203)
    int (attr='int', lineno=203)
    id (attr='TK_LPAREN', lineno=203)
globVarDecl (lineno=204)
    int (attr='int', lineno=204)
    id (attr='TK_RPAREN', lineno=204)
globVarDecl (lineno=206)
    int (attr='int', lineno=206)
    id (attr='ASCII_0', lineno=206)
globVarDecl (lineno=207)
    int (attr='int', lineno=207)
    id (attr='ASCII_9', lineno=207)
globVarDecl (lineno=208)
    int (attr='int', lineno=208)

```

```

        id (attr='ASCII_PLUS', lineno=208)
globVarDecl (lineno=209)
    int (attr='int', lineno=209)
    id (attr='ASCII_MINUS', lineno=209)
globVarDecl (lineno=210)
    int (attr='int', lineno=210)
    id (attr='ASCII_STAR', lineno=210)
globVarDecl (lineno=211)
    int (attr='int', lineno=211)
    id (attr='ASCII_SLASH', lineno=211)
globVarDecl (lineno=212)
    int (attr='int', lineno=212)
    id (attr='ASCII_LPAREN', lineno=212)
globVarDecl (lineno=213)
    int (attr='int', lineno=213)
    id (attr='ASCII_RPAREN', lineno=213)
globVarDecl (lineno=214)
    int (attr='int', lineno=214)
    id (attr='ASCII_SPACE', lineno=214)
globVarDecl (lineno=215)
    int (attr='int', lineno=215)
    id (attr='ASCII_TAB', lineno=215)
globVarDecl (lineno=216)
    int (attr='int', lineno=216)
    id (attr='ASCII_CR', lineno=216)
globVarDecl (lineno=217)
    int (attr='int', lineno=217)
    id (attr='ASCII_NL', lineno=217)
funcDecl (lineno=219)
    void (attr='void', lineno=219)
    id (attr='init', lineno=219)
    formals (lineno=219)
    block (lineno=221)
        exprStmt (lineno=221)
            ASSIGN (lineno=221)
                id (attr='EOF', lineno=221)
                number (attr='-1', lineno=221)
        exprStmt (lineno=223)
            ASSIGN (lineno=223)
                id (attr='ASCII_0', lineno=223)
                number (attr='48', lineno=223)
        exprStmt (lineno=224)
            ASSIGN (lineno=224)
                id (attr='ASCII_9', lineno=224)
                number (attr='57', lineno=224)
        exprStmt (lineno=225)

```

```

        ASSIGN (lineno=225)
            id (attr='ASCII_PLUS', lineno=225)
            number (attr='43', lineno=225)
    exprStmt (lineno=226)
        ASSIGN (lineno=226)
            id (attr='ASCII_MINUS', lineno=226)
            number (attr='45', lineno=226)
    exprStmt (lineno=227)
        ASSIGN (lineno=227)
            id (attr='ASCII_STAR', lineno=227)
            number (attr='42', lineno=227)
    exprStmt (lineno=228)
        ASSIGN (lineno=228)
            id (attr='ASCII_SLASH', lineno=228)
            number (attr='47', lineno=228)
    exprStmt (lineno=229)
        ASSIGN (lineno=229)
            id (attr='ASCII_LPAREN', lineno=229)
            number (attr='40', lineno=229)
    exprStmt (lineno=230)
        ASSIGN (lineno=230)
            id (attr='ASCII_RPAREN', lineno=230)
            number (attr='41', lineno=230)
    exprStmt (lineno=231)
        ASSIGN (lineno=231)
            id (attr='ASCII_SPACE', lineno=231)
            number (attr='32', lineno=231)
    exprStmt (lineno=232)
        ASSIGN (lineno=232)
            id (attr='ASCII_TAB', lineno=232)
            number (attr='9', lineno=232)
    exprStmt (lineno=233)
        ASSIGN (lineno=233)
            id (attr='ASCII_CR', lineno=233)
            number (attr='13', lineno=233)
    exprStmt (lineno=234)
        ASSIGN (lineno=234)
            id (attr='ASCII_NL', lineno=234)
            number (attr='10', lineno=234)
    exprStmt (lineno=236)
        ASSIGN (lineno=236)
            id (attr='TK_EOF', lineno=236)
            number (attr='256', lineno=236)
    exprStmt (lineno=237)
        ASSIGN (lineno=237)
            id (attr='TK_NUMBER', lineno=237)

```

```

        number (attr='257', lineno=237)
    exprStmt (lineno=238)
        ASSIGN (lineno=238)
            id (attr='TK_ADD', lineno=238)
            id (attr='ASCII_PLUS', lineno=238)
    exprStmt (lineno=239)
        ASSIGN (lineno=239)
            id (attr='TK_SUB', lineno=239)
            id (attr='ASCII_MINUS', lineno=239)
    exprStmt (lineno=240)
        ASSIGN (lineno=240)
            id (attr='TK_MUL', lineno=240)
            id (attr='ASCII_STAR', lineno=240)
    exprStmt (lineno=241)
        ASSIGN (lineno=241)
            id (attr='TK_DIV', lineno=241)
            id (attr='ASCII_SLASH', lineno=241)
    exprStmt (lineno=242)
        ASSIGN (lineno=242)
            id (attr='TK_LPAREN', lineno=242)
            id (attr='ASCII_LPAREN', lineno=242)
    exprStmt (lineno=243)
        ASSIGN (lineno=243)
            id (attr='TK_RPAREN', lineno=243)
            id (attr='ASCII_RPAREN', lineno=243)
    exprStmt (lineno=244)
        ASSIGN (lineno=244)
            id (attr='TK_EOLN', lineno=244)
            id (attr='ASCII_NL', lineno=244)

```

STDERR:

RETURN CODE: 0

Run ended on Wed Feb 25 00:51:51 2026

Custom Testing tool output

```

python test/run_tests.py lex ./main_lexer.py test/tests/lex
>> Test: basic_constructs.json passed!
>> Test: comment_EOL_check.json passed!
>> Test: comment_ending_in_EOF.json passed!
>> Test: comment_inside_&&_token_pair.json passed!
>> Test: comments_only.json passed!

```

```

>> Test: empty_file.json passed!
>> Test: illegal_Unicode_string_escape.json passed!
>> Test: illegal_characters.json passed!
>> Test: illegal_hex_string_escape.json passed!
>> Test: illegal_octal_string_escape.json passed!
>> Test: illegal_string_escape_character.json passed!
>> Test: illegal_whitespace.json passed!
>> Test: legal_strings.json passed!
>> Test: legal_whitespace.json passed!
>> Test: legit_token_salad.json passed!
>> Test: multi-line_string.json passed!
>> Test: non-7-bit_characters.json passed!
>> Test: string_containing_CR.json passed!
>> Test: string_containing_newline_closed.json passed!
>> Test: string_containing_newline_unclosed.json passed!
>> Test: string_escape_before_EOF.json passed!
>> Test: string_escape_before_newline.json passed!
>> Test: string_terminated_by_EOF.json passed!
>> Test: the_carat_schtick.json passed!
>> Test: whitespace_inside_&&_token_pair.json passed!
>> Test: whitespace_inside_||_token_pair.json passed!
>> Test: identifiers.json passed!
>> Test: digit_vs_identifier.json passed!
>> Test: comment_form_feed.json passed!
>> Test: string_form_feed.json passed!
>> Summary (lex): 30/30 tests passed
python test/run_tests.py parse ./main.py test/tests/parse
>> Test: global_variable_declarations.json passed!
>> Test: more_and_more_complex_global_declarations.json passed!
>> Test: the_main_problem.json passed!
>> Test: dangling_else.json passed!
>> Test: local_declarations_et_al.json passed!
>> Test: precedence.json passed!
>> Test: associativity.json passed!
>> Test: unary_minuses_so_many_unary_minuses.json passed!
>> Test: stringing_your_parser_along.json passed!
>> Test: formats_last_theorem.json passed!
>> Test: formalities_and_actualities.json passed!
>> Test: the_perfect_app.json passed!
>> Test: a_calculated_move.json passed!
>> Test: constructs_no_complex_expressions.json passed!
>> Test: constructs_complex_expressions.json passed!
>> Test: syntax_error?.json passed!
>> Test: syntax_error.json passed!
>> Summary (parse): 17/17 tests passed

```

Git Log

commit d301be734c40c8c01685952e3f3c6b7c6ec9e003
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 25 00:51:16 2026 -0700

add milestone2.md

commit e534bca37430efde8146d6d7ac9f2489b3508a03
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 25 00:18:42 2026 -0700

Removed unused pyright warning suppression comment

commit 598f834d39c649a10398460511f6f1e17abf5067
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 25 00:17:19 2026 -0700

Organize and document shaping rules

commit 45a327e00a666c345535c2ef117f5e5a94dc898d
Author: Avery Keuben <avery1516@gmail.com>
Date: Tue Feb 24 23:56:46 2026 -0700

Clean up parser by splitting functionality to different files

commit b0f051fa508b9df66d569e4ddecaecb06c02723
Author: Avery Keuben <avery1516@gmail.com>
Date: Tue Feb 24 23:04:09 2026 -0700

Move unary minus special casing to shaper subclass

commit f2599f48bd4c9c6b7ff6aa75ef2c6e520f9d8486
Author: Avery Keuben <avery1516@gmail.com>
Date: Tue Feb 24 09:58:07 2026 -0700

Add parser error handling

commit 3b38a8609b3fba90c759002ab819a265b75623fc
Author: Avery Keuben <avery1516@gmail.com>
Date: Tue Feb 24 09:33:48 2026 -0700

Fix missing parse.t02 test (syntax_error)

commit be075c4217011a83fcf06942eab1caa25f821e85
Author: Avery Keuben <avery1516@gmail.com>

Date: Mon Feb 23 14:12:41 2026 -0700

Fix test: unary_minuses_so_many_unary_minuses.j--

commit a5e4ba5993e50b8390786f79d33efc2b23a4b0c6

Author: Avery Keuben <avery1516@gmail.com>

Date: Sat Jul 16 10:59:04 2022 -0600

Almost fix teste a_calculated_move.j--: still need to fix unary minus

commit 53986b94dc1d12dbec208901aa34195a74114462

Author: Avery Keuben <avery1516@gmail.com>

Date: Sat Jul 16 10:45:12 2022 -0600

fix test syntax_error.j--

commit 9618c5a65e70f7de0da2e2f9c639a981dc727f9d

Author: Avery Keuben <avery1516@gmail.com>

Date: Sat Jul 16 10:37:12 2022 -0600

fix teset local_declarations_et_al.j--

commit edb2295076e533afb43b59d955d0f189f482dc27

Author: Avery Keuben <avery1516@gmail.com>

Date: Sat Jul 16 10:31:57 2022 -0600

fix test dangling_else.j--

commit 3dcef13a4b97f235dc3bd59b949ae2ee8fdec80d

Author: Avery Keuben <avery1516@gmail.com>

Date: Sat Jul 16 10:28:16 2022 -0600

fix test the_main_problem.j--

commit 698e40bf1bc3b4ecf610fad485aad1cf3a29e522

Author: Avery Keuben <avery1516@gmail.com>

Date: Sat Jul 16 10:26:41 2022 -0600

fix test global_variable_declarations.j--

commit f7ace13fa9da44800cf39bf6e2bb040f17b02c19

Author: Avery Keuben <avery1516@gmail.com>

Date: Sat Jul 16 10:24:47 2022 -0600

Add initial shaping

commit 4d83fe88006ba626b701123837c38d15356d0b82
Author: Avery Keuben <avery1516@gmail.com>
Date: Sat Jul 16 10:23:43 2022 -0600

Add testing for parser

commit c0b5d24ffe74af50e222ccbe04167634e44c5f2b
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 11 16:48:03 2026 -0700

Update flake.nix to allow for local visualizations though cpsc411.astview

commit 8a87aa45ef1da9b798c1b84a71d4b2e040d338c3
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 11 16:26:51 2026 -0700

Start using ASTShaper in cpsc411 lib

commit af84eccd5e347265e77767c61da3c58e2b9d3f12
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 11 16:06:22 2026 -0700

Add import for cpsc411 lib

commit acf672f8232c2f471e83c9ec4003b2a6b13c4d92
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 11 15:43:50 2026 -0700

Add grammar for parser

commit f2d406d4122a2bf8c859c6bfd862ea2c07b538bb
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:17:34 2026 -0700

Update milestone1.md

commit 6b3ad90829ac549ca3ff8d8acb5297eeff57b5bf
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:12:57 2026 -0700

Fix unknown file warning printing to stdout instead of stderr

commit c814502be78d7bf3df2c1324e9d25a2e4cc0c19b
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:10:32 2026 -0700

Update readme.md

commit 94a1b7bab21d9adf26302e87204887e2ad36ac9d
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:09:36 2026 -0700

Update readme.md

commit f51a7bf4098958831b2ba105cb58e7eb610a11bd
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:03:15 2026 -0700

Add milestone 1 document

commit 79ac01a93110afbb0b9a48e14087dc13f6257da7
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:02:13 2026 -0700

Fix crash when expected results file is missing for a partiular test in the unit test sy

commit 869ec93167ba19a9d6e927e9f98f8da49690ad07
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:01:25 2026 -0700

Remove unused Lexer paramerter in lexer.py

commit c200cfe036ead3dfe9a3a66fde9371bea4f7683f
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 14:00:50 2026 -0700

Add retun statements in main() to better show control flow

commit b5a62ddfed6ca2cf3b9ddf716405d855476fa2c
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 12:35:12 2026 -0700

Add error handling for unknown file argument

commit 54064d0292c3777558705097ac21471887251a84
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 12:31:46 2026 -0700

Add expected results for new tests

commit 429c6019fd49c3bfc97cf6afea1aa4f07a33dd0a
Author: Avery Keuben <avery1516@gmail.com>

Date: Fri Feb 6 12:18:53 2026 -0700

Add additional tests for form feed character and identifiers

commit bbf248f2fea5e6b0ee00da0ac00ebb3a1b1447c7
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 12:15:14 2026 -0700

Clean up grammar file for lexer using fragments & adjusting whitespace

commit 12cde92aad311722cc704fe6f04975c165d175c1
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 11:57:56 2026 -0700

Add type signatures to non-generated python files

commit a4c4f7ae9b72d2bed35c80c932c482f54eaa933a
Author: Avery Keuben <avery1516@gmail.com>
Date: Fri Feb 6 10:18:43 2026 -0700

Add wrapper around generated lexer

commit c1eee9a0434e5ff622b43c06d5f7b01bc1f27b3b
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 20:11:35 2026 -0700

Fix test: string_escape_before_newline

commit 7465386773a21ce9716a02f66575abfc937d1f5d
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 20:09:28 2026 -0700

Fix test: string_escape_before_EOF

commit ff782dc06e11ec16b673f148a9cc6d34fdab9007
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 20:08:02 2026 -0700

Fix test: string_containing_CR

commit f275c40ea4f49b3a97054a13c41d63cdde6f8f46
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 20:06:49 2026 -0700

Fix test: non-7-bit_characters.json

commit 76b7660acdb9cc2c46b0833ddfc8d5f06fe06d99
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:47:12 2026 -0700

Fix test: multiline_string

commit 98546d6bc971b60017e116ffdf57f904e433a241
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:43:07 2026 -0700

Fix test: comment_ending_in_EOF

commit 7991c6ab6a40b9b5b6e0ba752a2f32cf76077c53
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:39:41 2026 -0700

Fix test comment_EOL_check

commit 194dffabce5feb3b6e21da6f227a2c952a924d4d
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:39:09 2026 -0700

Add test to makefile

commit 0f41d8527424489a4268cdd10b4cd67de980eb55
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:26:50 2026 -0700

Make main.py executable

commit 0f359121beb88736db791a02ee120b81dd68cfe4
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:24:59 2026 -0700

Use stderr for logging warnings and errors. Limit number of warnings.

commit 6327d0604c95b6296154972b3bf3b9ebfededab2
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:24:00 2026 -0700

Fix test script using incorrect variable name

commit dd294389eaed3d5b3085f60d1947fc49216d1456
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 18:08:06 2026 -0700

Add expected results

commit 48ce0bf5b88e5d907a6968381c7d2663b17a676f
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 18:07:26 2026 -0700

Add provided tests

commit af118009bb7e93cdd8a872692ebf15454398a20a
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:16:30 2026 -0700

Add test script

commit 0b5e87a9de403221ccbae3139bf0713efd920ad7
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:15:20 2026 -0700

Generate expected test results and save as json

commit cc0641d7c975407a3ffa862d60a5975d01cd9ec2
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:14:10 2026 -0700

Fix typos in test system

commit ba217b14ff2a007e82f65afbec4fa966b8ce1f94
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:12:01 2026 -0700

Add default handler for unrecognized tokens to lexer grammar

commit 26598c5e10078ee7c180f59b230e0af5964528e0
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:07:35 2026 -0700

Add test for basic j-- language tokens

commit a730e71247a99edd3fdd30e316bea7a16b052170
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:07:08 2026 -0700

Add expected result generator for reference compiler

commit a911b82a99dcebfcd3045191466e92c9d5506478
Author: Avery Keuben <avery1516@gmail.com>

Date: Wed Feb 4 19:06:30 2026 -0700

Add basic testing framework

commit b05acbe9b0408f8d0a4775d030dc0c150ed7661e
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:05:26 2026 -0700

Add strings to lexer grammar

commit 55e26eef4fa387a2c3acd39e785512eb961d09bb
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 19:04:37 2026 -0700

Add keywords and operators to lexer grammar

commit 7ec4b69320dc2545a762fd1094cabde68e768d46
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 18:59:40 2026 -0700

Add basic token iterator from tutorial

commit c01a5d5bfeb7507bd6a48de03497ca7433ce983b
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 18:59:00 2026 -0700

Add basic logger

commit 975071f308bd9ce937d9ecd4708b7e23fba0fd6e
Author: Avery Keuben <avery1516@gmail.com>
Date: Wed Feb 4 18:58:33 2026 -0700

Add basic lexer generation

commit 0ee62aca52cb43e130d0047247926b49922f7ca3
Author: Avery Keuben <avery1516@gmail.com>
Date: Tue Jan 13 10:29:04 2026 -0700

Initial Project Setup

commit 1d598d92664734c0bc5123620aefbde808931afd
Author: Avery Keuben <avery.keuben@ucalgary.ca>
Date: Tue Jan 13 09:57:27 2026 -0700

Initial commit